# 面向 RISC -V 构建失败根因分析的日志压缩与规则学习方法

王泽华
中国科学院软件研究所
中国科学院大学
中国

## Abstract

近年来，RISC -V 依托开源、模块化与可扩展等特性，正在成为新一代通用与专用处理器的重要架构选择。大量开源软件包被迁移到 RISC -V 平台，同时也暴露出构建失败高发、原因复杂且随上游演进不断反复的问题。构建日志往往规模巨大、结构异构且噪声众多，真正决定失败的关键线索常被淹没在不同阶段、相距甚远的输出片段中，传统基于人工经验或规则匹配的排查方式效率低、成本高，难以支撑大规模迁移实践。

本文面向 RISC -V 软件包构建失败场景，提出一套“日志压缩 + 规则学习”的根因分析方法。首先，设计面向长程因果依赖的多粒度语义日志压缩流程（MGSC），通过显式错误锚点定位、基于实体的跨距离上下文回溯以及轻量语义压缩模型，在严格 *token* 预算下高保真保留与根因相关的关键信息。随后，提出基于历史修复经验的规则学习与根因推理方法（HRL-RCA），从历史修复记录中抽取结构化修复规则，构建按错误类型与构建阶段组织的规则树索引，结合压缩日志摘要进行条件匹配与推理，输出可解释的根因分类与修复建议。

基于来自实际 OBS 生产环境的 RISC -V 构建失败数据集的实验结果表明，所提出方法在关键错误行覆盖率、根因识别准确率以及修复建议质量等方面均优于多种基线方法，同时在日志压缩率和推理开销上保持良好平衡。

## Keywords

RISC -V, 构建失败, 根因分析, 日志压缩, 大语言模型

## 1 绪论

近年来，RISC -V 依托开源、模块化与可扩展等特性快速发展 [2, 3]，逐渐成为新一代通用计算与专用计算的重要选择方向之一。随着产业界与学术界对 RISC -V 的持续投入，大量主流开源软件正在向 RISC -V 架构迁移 [4]。然而由于主流开源软件的上游长期以 x86 和 ARM 为核心支持对象，其持续版本迭代往往不断引入新的架构强绑定特性，导致大量原本已适配 RISC -V 的软件包在跟随上游更新后再次构建失败。

在发行版级的软件包迁移实践中，开放构建服务（OBS）等平台通过自动化的构建、依赖管理与发布流程，为跨发行版、跨架构的软件包构建提供了统一基础设施。然而在 OBS 这类高度自动化的流水线中，构建失败呈现更高的频发性与多样性：同一软件包在不同架构、不同依赖版本、不同构建阶段（如 `%prep`、`%build`、`%install`、`%check`）均可能触发错误，且错误往往由架构差异、依赖链约束、构建脚本配置与补丁变更等多因素共同作用 [5, 6]。构建日志通常规模巨大（数万至数十万行），真正与失败相关的关键信息不足 5%，且日志格式在不同软件包与构建阶段间差异极大。更重要的是，很多时候最后出现的报错只是结果，真正触发失败的原因可能早在更前面的阶段就已出现，形成长程因果依赖。

为更直观地说明这种异步触发与长程因果依赖的特征，图 1 展示了 `aide` 软件包在真实构建环境中的失败日志片段。该构建错误在日志中呈现出典型的跨距离分布形态：（1）构建终止信号（末端）：构建系统最终在第 1021 行抛出笼统的构建终止信号（`RPM build errors`）。（2）显性错误语句（中端）：向前回溯，真正的编译期错误出现在第 998 行，日志提示 `` `GCRY_MD_SM3' undeclared `` （宏未定义）。（3）隐性根本原因（前端）：然而，导致该宏未定义的根本原因隐藏在数百行之前的依赖安装阶段——在第 471 行，系统安装了 `libgcrypt-1.8.7-1.oe1` 版本，而该版本不支持 SM3 算法。值得注意的是，第 471 行仅是一条普通的 `installing` 常规输出，完全没有任何异常或错误标记。

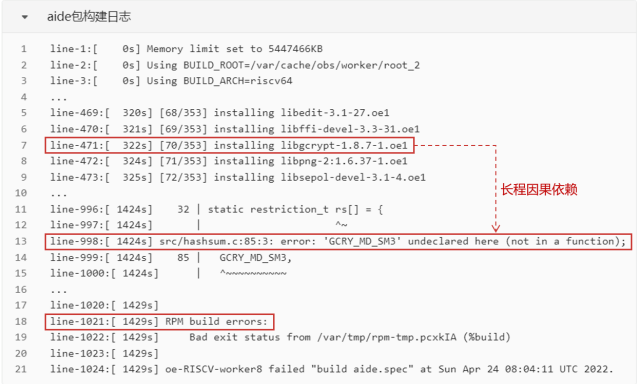


**Figure 1:** 构建日志中的长程因果依赖示例

这种原因在前、错误在后的现象揭示了处理超长构建日志的核心痛点：如果自动化分析仅依靠截取报错附近的局部窗口，或者单纯依赖 `error` 等异常关键字进行过滤，第 471 行这部分决定性的版本依赖线索将被彻底丢弃。这正是本文面临的挑战一：如何从超长日志中找准真正的错误——关键报错被大量噪声淹没，真正触发失败的原因可能早在数百行前出现，仅截取报错附近的窗口容易错过因果线索。

即便日志压缩阶段成功保留了关键证据链，根因分析与修复推荐仍然面临缺乏可靠历史经验的问题。基于向量相似度的朴素 RAG 容易产生相似误导：相似日志并不等价于适用修复。如图 2 所示，`apache-mime4j` 软件包构建日志与 `hibernate` 软件包构建日志中出现完全一致的关键报错，但进一步结合各自的构建脚本与环境约束后可以发现，其有效修复方案并不相同。该案例说明：构建失败的修复方案并不仅取决于报错文本本身，还强依赖于包的构建阶段、工具链与依赖组合等上下文前提。仅以文本相似度进行历史案例检索，容易将不满足适用前提的历史案例引入上下文，反而放大误判风险。

此外，大模型在此类任务上的主要问题并非缺少语言表达能力，而是推理结果的可验证性不足。面对多证据并存、证据片段前提缺失或证据互相矛盾的情况，模型倾向于生成连贯但难以复现的解释与方案。在工程排障中，修复建议必须能够被复查与复现：不仅要给出原因，还应明确基于哪些日志证据、满足哪些前提条件、对应哪条历史修复记录。这正是本文面临

**Figure 2:** 相似报错但不同修复方案的误导性示例

的挑战二：如何从历史案例中找对能用的修复参考——修复经验分散在不同仓库的提交记录与 issue 讨论中，相似日志并不等价于适用修复，版本/依赖/阶段不同时可能产生误导。

面向 RISC-V 软件包迁移的构建失败分析，亟需一种能够在有限 token 上下文预算下保留高保真证据链、并将历史修复经验以可判定条件与可追溯证据的方式组织起来的自动化方法。

针对上述挑战，本文做出以下贡献：

(1) 提出面向长程因果依赖的多粒度语义日志压缩方法（MGSC），通过"显式错误锚点定位—基于实体的跨距离上下文回溯—轻量模型细粒度语义压缩"三阶段流水线，在严格 token 预算下高保真保留关键证据链。
(2) 提出基于历史修复经验的规则学习与根因推理方法（HRL-RCA），将修复经验抽取为结构化规则并按"错误类型—构建阶段—适用前提"组织为树形索引，在线推理时通过条件路由召回满足前提的规则，结合 ReAct 范式生成可追溯的根因分析与修复建议。
(3) 基于 OBS 生产环境构建 RISC-V 构建失败数据集并开展系统实验，结果表明 MGSC 在关键证据覆盖率上优于现有压缩方法，HRL-RCA 在综合诊断得分上达到 79.0%，显著超过多种基线，同时处理耗时降低约 61%。

## 2 相关工作

### 2.1 日志解析与语义压缩

日志解析旨在将非结构化日志归一化为事件模板与动态参数的结构化表示。早期方法依赖规则与正则匹配，典型如 SLCT[7] 通过聚类算法挖掘日志模式、Drain[8] 通过固定深度解析树实现在线高效解析，Logram[9] 利用 *n*-gram 字典加速模板识别。随着深度学习的发展，基于 LSTM 的方法 [10] 学习正常执行路径的时序规律以检测异常；基于 Transformer 与自监督学习的方法获得更强的语义表征能力 [11]。然而这些方法在跨系统、跨版本迁移时面临泛化不足的问题。

近年来，大语言模型（LLM）为日志解析提供了新的范式 [12]。基于 prompt 的少样本日志解析 [13] 通过示例选择降低调用成本；DivLog[14] 通过上下文学习提升跨域解析能力；面向可解释性的在线日志分析方法 [15] 通过提示策略增强 LLM 的日志理解能力。面向 CI/CD 场景的日志分析框架 [16] 提出 token 高效的预处理与结构化诊断流程。然而在软件包构建失败这类长程因果依赖强、阶段跨度大的场景中，如何在有限 token 预算下稳定保留可追溯证据链，仍需更贴合任务特性的压缩方法。

### 2.2 基于日志的根因分析

根因分析（RCA）旨在从系统输出中定位故障的底层原因。传统方法通过规则关联完成事件匹配 [17]，或结合源码分析推断失败执行路径 [18]。深度学习方法将根因分析表述为日志行相关性排序任务，从日志中抽取故障指示性描述与参数用于结构化诊断 [5]。

LLM 驱动的 RCA 更强调直接从故障日志出发生成可读的诊断结论。RCAgent[1] 通过工具增强的 LLM 智能体实现自主诊断，基于 Agent 的设计让模型通过多步推理逐步缩小根因范围。基于检索增强的方法 [6] 将多源信息组织为 LLM 可消费的上下文完成排障建议。然而，仅依赖"相似日志检索 + 自由生成"容易引入误导证据或放大幻觉，必须把适用前提、证据对齐与验证机制纳入体系化方案。本文方法正是从历史修复经验中学习可判定规则，并以树形索引实现条件路由，从源头降低误导风险。

### 2.3 构建失败分析与软件包迁移

除通用日志分析与 RCA 之外，已有工作也面向软件构建失败本身展开研究。一类研究聚焦于大型开源生态的构建失败统计与模式挖掘 [2–4]，通过对发行版仓库或持续集成流水线中的失败记录进行汇总，给出典型失败类型、易错阶段以及与依赖关系、配置脚本之间的相关性分析，为后续自动化修复提供经验基础。另一类工作则从工程视角出发，探索在 OBS、GitHub Actions 等平台上利用规则库或分类模型对构建失败进行自动归因与分派 [5, 6]，降低维护者人工排查的工作量。

然而，这些针对构建失败的研究往往仍停留在"短窗口 + 局部特征"的范式上：要么依赖人工设计的错误模式和启发式规则，难以应对异构日志和跨版本演进；要么在单次调用中直接将长日志截断输入模型，忽略了跨阶段的长程因果依赖。此外，现有系统多以错误类别预测或简单的经验推荐为目标，缺乏对历史修复知识的结构化组织和可判定适用前提的建模。相比之下，本文在构建失败场景下同时引入面向证据链保留的多粒度日志压缩（MGSC）以及基于规则树的历史修复知识建模（HRL-RCA），在统一框架内兼顾了长日志处理效率、跨阶段根因定位能力与修复建议的可验证性。

## 3 方法

本文采用两阶段流水线处理 RISC-V 构建失败。第一阶段（MGSC）对超长日志进行多粒度语义压缩，输出以关键错误行为中心、包含必要上下文的结构化摘要。第二阶段（HRL-RCA）以压缩摘要为输入，结合历史修复经验的规则树索引，通过条件路由与 ReAct 推理生成可追溯的根因分析与修复建议。

### 3.1 MGSC：多粒度语义日志压缩

构建失败日志的压缩并非简单删掉大部分行，而是要在有限预算内尽可能保留能支撑根因分析的关键证据。为此，本文提出面向长程因果依赖的多粒度语义日志压缩方法（MGSC），其核心思想是以显式错误信息为锚点快速定位失败表征，再以关键实体为线索跨距离回溯补齐触发条件，最后在候选证据集上进行细粒度语义压缩与结构化整理，从而把原始长日志转化为不丢失因果信息的摘要结果。该方法强调多粒度与长程关联两点：多粒度体现在从关键错误行到错误相关片段再到实体级相关线索的逐层聚焦；长程关联则体现在通过实体回溯把相距很远的因果线索重新拼接，避免仅截取尾部报错导致的信息丢失。如图 3 所示，MGSC 分为三个阶段：阶段 1 通过轻量规则与启发式评分，优先保留错误附近和阶段关键转折点对应的片段，追求高召回的候选错误块；阶段 2 从候选错误块中抽取关键实体线索，在全日志范围内跨距离回溯补全因果链；阶

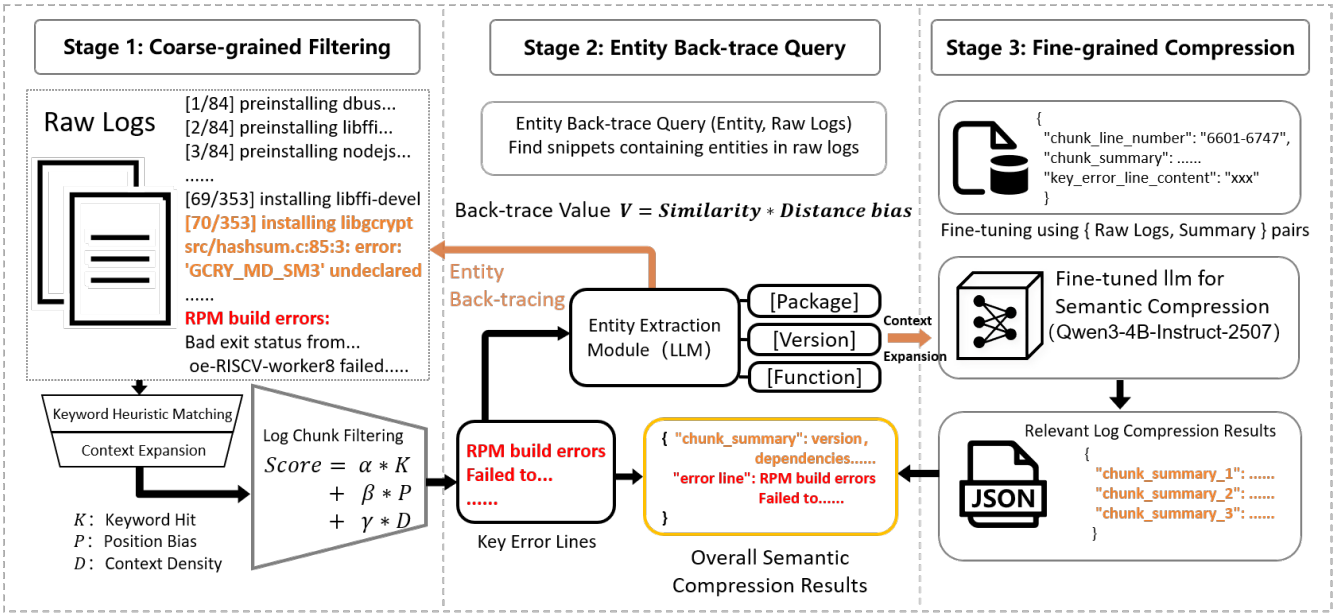


**Figure 3: MGSC** 多粒度语义日志压缩方法整体流程

段 3 在候选片段集合上执行细粒度语义压缩与结构化整理，输出固定预算内的高密度信息载体。

给定原始构建日志 $L = \{l_1, l_2, \ldots, l_N\}$，MGSC 的目标是生成满足 token 预算约束 $|S| \le B$ 的结构化压缩结果 $S$，使 $S$ 中保留的关键证据行与因果线索最大化。其中 $B$ 通常取下游大模型上下文长度的一半，为后续根因推理与工具调用预留充足空间。

*3.1.1* 快速粗粒度过滤. RISC-V 软件包在 OBS 平台上的构建日志通常具有超长、异构、噪声高的典型特征：一方面，日志中包含大量正常编译输出、平台提示与工具链信息，真正与失败根因相关的内容只占极小比例，整体信噪比极低；另一方面，构建流程跨越多个阶段，错误触发条件可能出现在很早的探测或依赖检查阶段，却在后续编译或链接阶段以显式报错形式集中爆发。若直接对全量日志进行实体回溯或语义压缩，不仅计算开销高、token 成本难以控制，还容易被冗余信息干扰。因此，MGSC 首先设置快速粗粒度过滤阶段：用轻量规则与启发式策略快速剔除已验证无用的冗余输出，定位显式错误锚点，并以锚点为中心生成少量高相关候选日志块。

具体而言，首先通过正则规则集合 $R_{drop}$ 去除跨包稳定出现但与失败无关的冗余输出（如编译器版本回显、下载进度、固定格式状态提示等），得到过滤后日志 $L' = \{l_i \in L \mid g(l_i) = 1\}$。随后通过分层关键词匹配召回关键错误行集合 $A$，匹配层次包括：（1）强错误模式——`error:`、`fatal`、`undefined reference`、`No such file`、`Bad exit status` 等；（2）构建终止提示——`RPM build errors`、`make: *** Error` 等；（3）补充召回模式——`undefined symbol`、`cannot find -lxxx` 等弱信号。对重复报错与相同错误模板进行合并去噪，避免同一错误占据过多预算。

在获得关键错误行集合 $A$ 后，对每条错误行做前后窗口扩展生成候选日志块 $\mathcal{B}$，以保证局部语义完整。对块内每行 $l_i$ 按以下行级评分函数衡量重要性：

$$S(l_i) = \alpha \cdot I_{key}(l_i) + \beta \cdot f_{pos}(i) + \gamma \cdot D_{context}(i) \tag{1}$$

其中 $I_{key}$ 为关键词匹配指示函数，$f_{pos}(i) = e^{-\lambda(N-i)/N}$ 为位置偏置项（越靠近日志末尾权重越高），$D_{context}$ 为局部关键词密度项。块级得分取块内行得分的聚合（最大值 + 平均值），以保证含有强错误锚点的块能够优先上浮，同时避免块仅因更长而占据优势。系统按块级得分从高到低排序，在 token 预算约束下选择 Top-n 块输出。本文约束该阶段输出的总 token 长度不超过下游大模型上下文窗口的四分之一，为后续实体回溯补全与语义压缩预留充足空间。

*3.1.2* 基于实体的上下文回溯. 经过快速粗粒度过滤后，系统已从超长日志中获得一组高相关的候选日志块，并以显式错误行作为锚点保留了失败发生时的局部语义上下文。然而在 RISC-V 软件包构建失败场景中，单纯依赖错误附近的窗口片段往往仍不足以支撑稳定的根因分析。原因在于构建失败常呈现明显的跨距离特征：真正触发失败的线索可能出现在更早的 `configure` 探测、依赖安装或配置信息中，而最终错误却在 `%build` 阶段集中爆发；同一根因也可能在多个阶段以不同形式被反复暗示，例如先出现找不到头文件/库的提示，随后在编译阶段演化为类型/符号未定义，在链接阶段进一步表现为 `undefined reference`。这种原因在前、现象在后的长程因果依赖，使得仅保留显式错误附近片段容易出现证据断裂。

为实现跨距离因果链的补全，本阶段从粗粒度过滤得到的显式错误片段中抽取可用于回溯的查询要素，并将其组织为结构化实体集合 $E$。这里的实体并非泛指自然语言中的任意名词，而是指能够在构建日志中稳定出现、能够指向失败链条关键节点的符号化线索，具体包括：函数/符号名（如 `__cpuid`、`riscv_v_asm_flag`）、依赖库名（如 `libatomic`、`openssl-devel`）、缺失头文件名、目标文件与路径、构建目标、宏定义与关键编译/链接选项等。实体集合的质量直接决定回溯的有效性：实体过少会导致回溯召回不足，实体过杂则会引入大量无关片段。

以实体集合 $E$ 为查询条件，在原始日志 $L$ 中检索相关片段。对每行 $l_j$ 定义回溯价值：

$$V(l_j) = \max_{e \in E} \left( \mathrm{Sim}(l_j, e) \cdot \frac{1}{\log(1 + \lambda \Delta t(l_j, e))} \right) \tag{2}$$

其中 Sim 表示模糊匹配相似度（支持子串匹配、大小写不敏感匹配与简单编辑距离），距离项保证在相似度相当时偏向更近的行。系统选取回溯价值高于阈值的命中行，并聚合为回溯片段，将分散在不同构建阶段的因果证据补齐。例如，当显式错误中出现 `undefined reference to `foo_v_probe'`，实体回溯会在日志前部 `configure` 阶段找到相关的 `checking for foo_v_probe... no` 以及对应的 `config.log` 中的编译失败信息，从而将配置探测失败与链接错误之间的因果关系补全。

*3.1.3* 细粒度语义压缩. 经过粗粒度过滤与实体回溯后，系统已收敛出两类高价值信息：以显式报错为核心的关键错误行及其局部上下文，以及围绕错误实体在全日志范围内回溯得到的相关片段。然而，回溯得到的相关信息仍以原始日志片段形式存在，内容可能冗余、表达不统一，且长度较长，易超过下游推理模型可承受的上下文预算。如果直接将这些片段拼接输入大模型，不仅 token 成本高，而且噪声与重复信息会干扰后续根因分析，使推理结果不稳定。

为此，本阶段引入细粒度语义压缩：使用领域微调的轻量模型对回溯片段进行摘要与要点凝练，并与粗粒度阶段保留的显式错误信息融合，生成结构化、预算可控的日志语义压缩结果。这里的摘要并非泛化叙述，而是围绕根因分析需求进行任务导向的信息整合：一方面保留对定位有用的上下文行/片段摘要，另一方面给出阶段归属标签（如 `%prep`、`%build`），同时凝练可直接用于判断根因方向的关键事实，例如缺少某依赖导致 `configure` 禁用特性、链接阶段找不到 `-lxxx`、某符号在 RISC-V 下不可用等。

基座模型选用 Qwen3-4B-Instruct，该模型在指令跟随与文本生成方面具备良好基础，同时参数规模适中，便于在工程系统中以较低成本部署推理。通过面向任务的监督微调，将模型能力收敛到保留关键信息、压缩无用细节并且输出格式稳定的目标上。微调训练集包含 2925 条高质量日志块语义压缩数据，采用 SFT + LoRA 参数高效微调策略。实验表明，微调后模型的压缩比由 19.66× 提升至 30.6×，同时关键错误覆盖率从 74.3% 提升至 75.7%，说明领域微调在提升压缩效率的同时保持了关键信息的保留能力。

最终将细粒度摘要与粗粒度阶段的显式错误信息融合，输出结构化压缩结果 $S$，使其既可读、可复现，也更利于后续基于历史修复知识的根因推理。

## 3.2 HRL-RCA：基于历史修复经验的规则学习与根因推理

为降低朴素相似检索带来的相似误导，并使修复建议具备可验证的证据依据，本文提出基于历史修复经验的规则学习与根因推理方法（HRL-RCA）。其基本思想是：从历史提交记录中抽取并显式结构化 "症状—适用前提—修复动作—证据" 之间的对应关系，将分散的修复经验沉淀为可复用的修复规则，并进一步以问题类别、构建阶段与可判定条件为主线构建可维护的树形索引。在线推理时不再依赖相似度直接召回，而是以 MGSC 得到的语义压缩结果为输入进行条件路由检索，在满足适用前提的路径上命中并召回相关规则，再在证据约束下生成根因解释与修复方案，从而实现可解释、可追溯、可执行的输出。

如图 4 所示，整体流程分为离线构建与在线推理两部分。离线阶段面向历史修复记录，首先从 commit message、diff/patch 以及与打包相关的 spec 变更中抽取并结构化修复经验，将其统一沉淀为可复用的修复规则；随后依据错误大类与构建阶段构建树形索引骨架，并从修复规则中提取可判定条件作为分支依据，将每条规则挂载到满足其适用前提的叶节点，同时引入冲突检测与增量更新机制。在线阶段以 MGSC 输出的压缩摘要 $S$（可选附加 spec/patch 与环境摘要）为输入，先完成错误类型与构建阶段判定以定位索引入口，再沿条件分支进行匹配，召回候选叶节点中的修复规则；最终在证据约束下生成包含 "根因—修复—适用前提—证据引用" 的诊断报告。

*3.2.1* 历史修复信息结构化. 历史修复经验在工程实践中主要以提交记录形式沉淀，包括 commit message、代码 diff/patch 以及与打包相关的 spec 变更等。这类信息虽然包含大量可复用的修复线索，但天然是非结构化、表述不统一且证据粒度不一致的：同一类问题可能被描述为不同措辞（如 "fix build"、"riscv workaround"、"linker error" 等），而真正可执行的修复动作往往隐藏在 diff 片段中。若直接将原始 commit 用作相似检索或文本召回对象，容易再次引入相似误导，并且难以显式表达适用前提与修复动作之间的绑定关系。因此，本方法首先将历史提交经验抽象为结构化单元——修复规则，把经验从文本提升为可计算、可路由、可维护的知识条目。

修复规则的构建以单条修复提交为基本对象，输入包含：（1）commit message：描述意图与背景；（2）diff/patch：具体代码或脚本修改内容；（3）spec 相关变更：依赖声明、宏与构建参数、补丁引用等打包侧修改；（4）关联 issue/PR 讨论：问题复现、适用前提与边界条件等补充证据。输出为一个结构化 JSON 记录，字段设计强调两点：一是显式编码适用前提（错误类型 `error_type`、构建阶段 `build_stage`、触发条件 `trigger_condition`、是否为 RISC-V 特有 `risc_v_specific`、是否可泛化 `can_generalize`），以支持后续树形索引的分支构建与条件路由；二是保留证据载体（`files_touched`、`commit_id`、`notes`），保证生成结果可追溯、可复核。

在构建流程上，采用 "传统解析预处理—大模型结构化—校验与规范化" 三级流水线。首先，使用传统方法对提交元信息与改动内容进行初步解析，包括对 commit message、diff/patch、spec 变更以及关联 issue/PR 文本进行分段与要素抽取（如 touched files、关键变更块、依赖声明与宏参数修改等），以降低大模型输入噪声并提供必要的结构提示。其次，基于统一的提示模板与固定输出 schema，调用大语言模型将上述解析结果结构化为修复规则，输出字段覆盖 `error_type`、`build_stage`、`trigger_condition`、`root_cause`、`fix_action` 等，并要求将触发条件、修复模式等关键字段尽可能映射到预定义的规范化标签集合。最后，引入独立的校验模块对大模型输出进行自动校验与纠偏：一方面检查修复规则是否满足结构化 schema 约束（字段完整性、类型一致性、必填项非空等），另一方面对 `fix_action`、`trigger_condition` 等字段进行规范化一致性检查。当校验失败时，系统触发重写策略（要求模型按校验反馈重新生成特定字段），保证进入知识库的修复规则具有稳定格式、可路由前提与可追溯证据。

*3.2.2* 修复规则树形索引构建与维护. 在得到结构化的修复规则后，需要将分散的修复经验组织为可路由、可维护的知识结构。本文采用树形索引作为知识组织形态，理由有三：其一，构建失败的人工诊断过程本质上遵循先判别错误类别与构建阶段，再依据适用前提逐步缩小范围的排查逻辑，树形索引以显式的条件分支刻画这一过程，使规则匹配路径天然可解释、

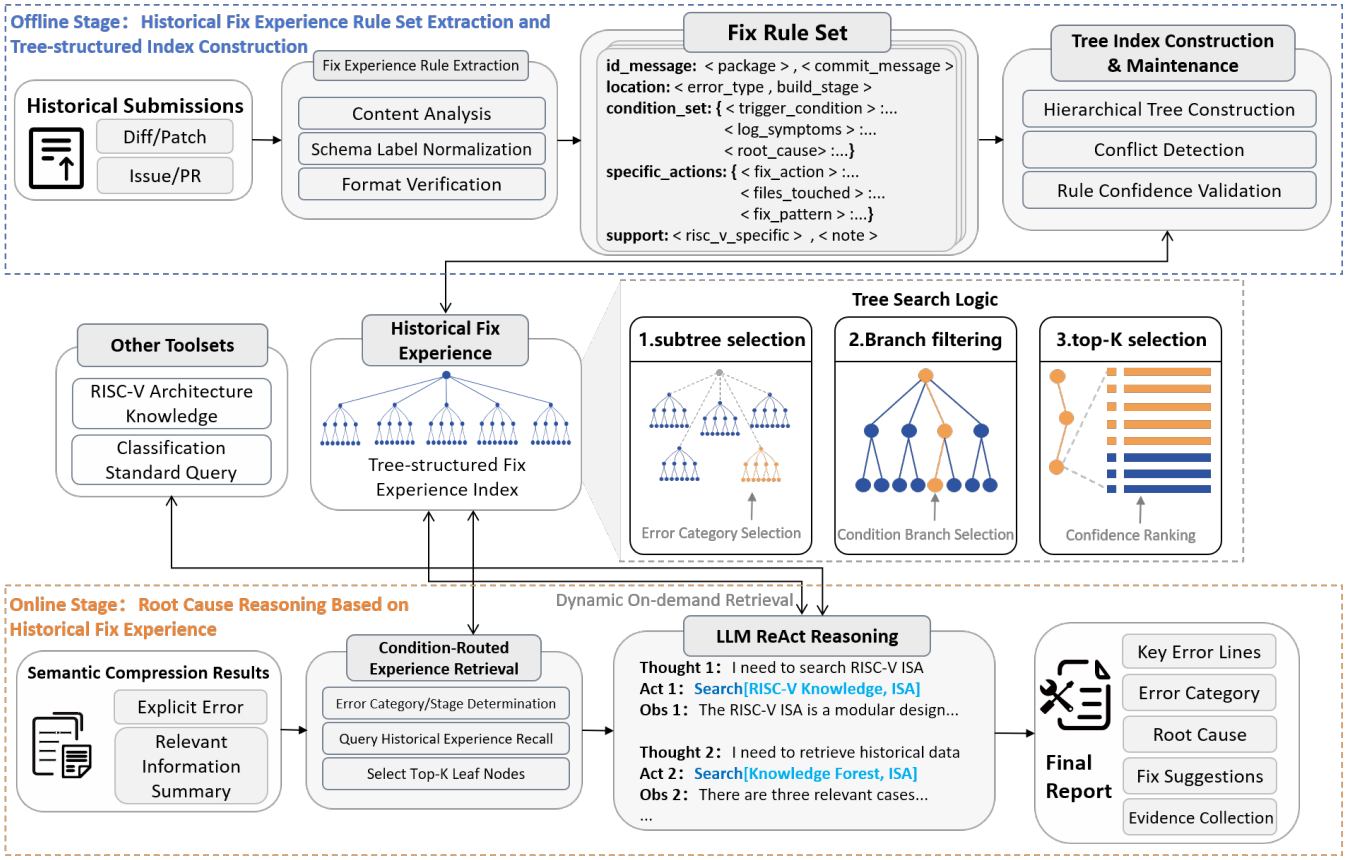


**Figure 4: HRL-RCA** 基于历史修复经验的规则学习与根因推理框架

可复查；其二，修复经验的有效性高度依赖前提约束（版本、架构、依赖状态、构建系统与阶段等），树形索引的路由过程要求每一步都依据可判定条件进行分支查找，从而在检索入口就完成可用性过滤，从源头弱化相似度检索带来的错误召回风险；其三，若将全部规则及其条件直接交由大模型做条件检索，token 成本不可控，且模型易在冗余信息中出现注意力分散与条件遗漏，树形索引通过分层固化路由结构，把筛选过程前置为结构化剪枝，仅将少量满足前提的规则注入模型，实现小上下文、强约束、可复查的推理输入。

如图 5 所示，索引以树形根节点为入口，第一层按本文的错误分类体系划分为五个问题域（架构兼容、版本兼容、依赖配置、编译配置与其他错误），使在线推理能够先在问题域层面完成粗路由；第二层按构建阶段组织（如 `%prep`、`%build`、`%install`、`%check` 等），将历史经验约束在一致的阶段语境中，避免同一报错文本在不同阶段含义不同带来的误判。更深层节点由可判定的路由条件驱动分叉，这些条件来自修复规则中显式编码的 `trigger_condition`、`log_symptoms` 等信息，例如“是否探测到依赖 xxx 缺失”、“是否仅在 RISC-V 架构下触发”等。叶节点挂载具体修复规则集合，并绑定 `fix_action`、适用前提与必要备注，在线推理一旦命中路径，便可同时获得“前提—动作—证据”的整体化结果。

为支持持续演进，设计增量更新与冲突检测合并机制。增量更新以新增修复规则为输入，首先完成与离线构建一致的规范化处理，并据此确定其路由位置。当新增规则的条件集合

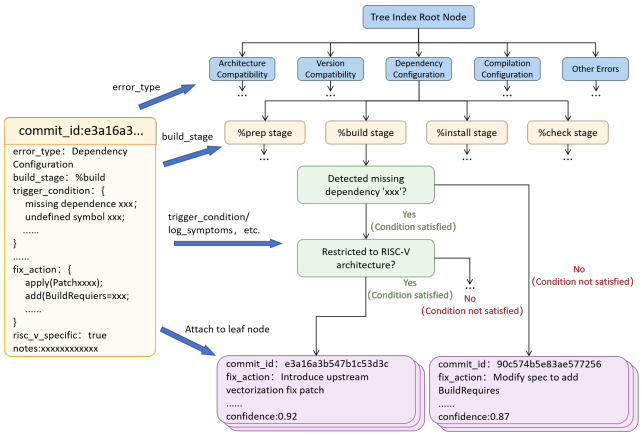


**Figure 5:** 修复规则树形索引结构示意

$C_{new}$ 与已有规则 $C_{old}$ 存在包含关系时：若 $C_{new} \supset C_{old}$，说明新规则引入了更严格的判定条件，系统在树中新增更深一层的分支节点，将细规则优先匹配；若 $C_{new} \subset C_{old}$，说明新规则覆盖范围更广，系统进行泛化调整以减少重复分支。上述处理避免了简单并存堆叠带来的路径膨胀。

同时引入规则置信度进行质量控制：

$$\mathrm{Conf}(r) = \alpha \cdot S_{evi}(r) + (1-\alpha) \cdot S_{time}(r) \tag{3}$$

其中 $S_{evi}(r) = n_r/(n_r + k)$ 为证据支持度（$n_r$ 为支撑记录数，$k$ 为平滑常数，默认 $k$=3），$S_{time}(r) = 1/(1 + (t_{now} - t_r)/T)$ 为时间新鲜度（$T$=90 天，即规则更新 90 天后新鲜度降至 0.5）。当 $\mathrm{Conf}(r)$ 低于预设阈值时，规则仅作为低优先候选参与在线推理排序，并进入待人工审核队列；达到阈值时才被确认为可路由的高优先级叶节点证据，从而在增量更新中实现对知识质量与时效性的联合约束。

*3.2.3* 在线根因推理. 在线推理阶段的目标是：给定一次构建失败任务的结构化输入，沿树形索引完成条件路由与历史经验召回，并在证据约束下生成可解释、可执行、可验证的根因分析与修复建议。本方法采用证据驱动的 ReAct 推理框架组织在线流程：以 MGSC 输出的日志语义压缩结果 $S$ 作为推理入口，将外部知识查询与树形索引检索封装为可调用工具，使模型在推理过程中以 "行动 → 观察 → 再推理" 的方式逐步补全适用前提并收敛到可用证据。

错误类型与阶段判定。在线推理的第一步是判定本次失败属于哪一类错误、发生在构建流程的哪个阶段。系统以 MGSC 输出的压缩结果为入口，结果中已包含显式错误行、阶段相关线索、关键实体以及若干与错误相关的证据片段。阶段判定优先使用压缩结果中的显式字段；若未显式给出，则结合错误锚点附近的典型输出特征进行推断。错误类型判定以压缩结果中的错误锚点与关键实体为主要依据，系统查询预先定义的五大类错误标准及其典型触发条件，将当前错误现象映射到候选类别；同时，当压缩结果中出现与架构强相关的实体或模式时，调用 RISC-V 相关知识查询工具进行补充判断，以避免仅凭表面报错做出误判。

相关历史修复经验查询。完成错误类型与阶段判定后，系统进入树形索引检索环节。首先根据 `error_type` 定位到对应的问题域节点，再根据 `build_stage` 进入阶段子树，将候选空间限定在语义一致的局部范围内。进入阶段子树后，以压缩结果中的触发条件信息、关键实体等作为路由依据，与索引中各分支条件进行匹配。匹配过程采用自顶向下遍历：当线索能够明确满足某一分支条件时沿分支下行；当线索不足时保留该分支为候选以避免过早剪枝；当线索明确不满足时跳过该分支。当检索得到多个候选叶节点时，系统进行轻量排序：排序依据由条件匹配程度（触发条件与关键实体覆盖度）与规则置信度两类信号构成，排序后输出 Top-K 条修复规则作为候选证据集合。若未能命中任何叶节点，系统回退至更上层节点获取更泛化的规则集合。

基于 **ReAct** 范式的推理。在获得召回的历史修复经验后，系统进入最终答案生成阶段。系统将三类信息注入上下文：其一是 MGSC 输出的压缩结果 $S$，提供关键显式错误、阶段线索与实体证据；其二是历史修复经验库的检索结果（排序后的 Top-K 修复规则）；其三是检索路径信息（问题大类、错误阶段、条件分支序列）。模型按 ReAct 流程进行推理：首先基于压缩结果与候选经验形成初步假设，若发现关键信息缺失或候选之间存在不一致，模型以 "行动" 的形式调用工具进行补充查询（如查询 RISC-V 相关知识或验证设想修复方案），工具返回作为 "观察" 结果写回上下文，模型据此修正假设并收敛。最终系统要求模型输出结构化诊断报告，包含：（1）关键错误信息与阶段定位；（2）错误类型判定及分类依据；（3）可复核的根因解释；（4）具体可执行的修复建议；（5）支撑证据集合与历史修复记录引用。若检索到多条候选经验且均具备一定合理性，系统输出主推荐与备选方案，并用 `trigger_condition` 说明各自的适用边界，避免在证据冲突时给出不可验证的折中结论。

# 4 实验评估

## 4.1 实验设置

*4.1.1* 数据集. 实验数据来自 OBS 真实生产环境下的 RISC -V 软件包构建失败日志。日志压缩实验使用 133 个不同软件包的构建日志，总计 149219 行日志文本，并由专家标注 4582 行关键修复相关日志行作为评测基准。根因分析实验使用 174 个典型构建失败案例，覆盖五种错误类型（架构兼容、编译配置、依赖配置、版本兼容、其他错误）。

如图 6 所示，数据集中版本兼容类错误占比最高（29.9%），编译配置与依赖配置各占 23.6%，其他错误 14.4%，架构兼容 8.6%。从构建阶段分布看，`%build` 阶段占 50.0%，`%prep` 占 24.1%，`%check` 占 20.7%，`%install` 占 5.2%。

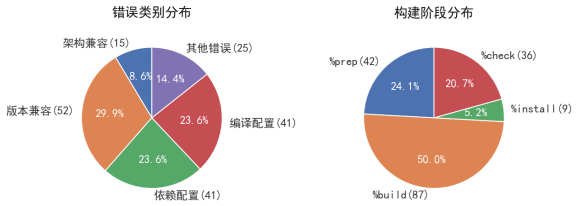


**Figure 6: RISC -V** 构建失败数据集错误类别与构建阶段分布

*4.1.2* 基线方法. 日志压缩实验选取三种基线：（1）LogSage[16]——面向长日志的 LLM 驱动分析框架；（2）TemplateRAG——模板化解析与检索增强方法；（3）LogPrompt[15]——通过提示工程增强 LLM 的日志分析能力。

根因分析实验设置四类对比基线：（1）朴素 RAG Agent——基于语义相似度的经典检索增强生成；（2）LogSage[16]——改进的多路 RAG 检索增强；（3）MCTS——搜索式推理框架；（4）旗舰推理模型（Seed1.8、Kimi K2、DeepSeekV3.2）——以模型能力为主要提升来源的对照。

*4.1.3* 评价指标. 日志压缩以行级 Precision/Recall/F1 评估关键证据保留能力。根因分析采用综合得分函数：

$$\mathrm{score}(i) = \frac{s_i + c_i + r_i + f_i}{4} \tag{4}$$

其中 $s_i$ 为关键错误语句提取得分，$c_i$ 为错误分类得分，$r_i$ 为根因解释得分，$f_i$ 为修复建议得分。

## 4.2 日志压缩实验结果

如表 1 所示，MGSC 的 Recall 为 0.668，显著高于 LogSage（0.574），说明显式错误锚点粗筛与基于实体的回溯补全能够更有效地找回长程因果链上的关键线索。同时 MGSC 的 Precision 远高于 TemplateRAG（0.302 vs 0.047），表明在提升覆盖率的同时仍能抑制大量无关噪声。在性能方面，MGSC 在三组日志上的平均输出 token 为 3442/4234/5502，始终处于几千 token 量级，相比 TemplateRAG（38746/49883/55406）小一个数量级，为后续推理预留了充足上下文预算。运行耗时方面，MGSC 在超长日

**Table 1:** 日志压缩方法准确性对比

| 方法 | Prec | Rec | F1 |
|---|---|---|---|
| LogPrompt | 0.260 | 0.300 | 0.334 |
| TemplateRAG | 0.047 | 0.769 | 0.080 |
| LogSage | 0.308 | 0.574 | 0.387 |
| MGSC（本文） | **0.302** | **0.668** | **0.389** |

志下相比 LogPrompt 与 TemplateRAG 可实现约 2.3–3.6× 的加速。

## 4.3 MGSC 消融实验

为验证各模块贡献，对 MGSC 进行消融实验。

**Table 2: MGSC** 消融实验结果

| 配置 | Prec | Rec | F1 |
|---|---|---|---|
| MGSC（完整） | 0.302 | 0.668 | 0.389 |
| w/o Anchor Filter | 0.274 | 0.352 | 0.328 |
| w/o Entity Backtracking | 0.292 | 0.584 | 0.372 |
| w/o Semantic Compressor | 0.246 | 0.668 | 0.349 |

如表 2 所示，去除错误锚点粗筛（w/o Anchor Filter）后 Recall 由 0.668 降至 0.352（↓47.3%），F1 下降 15.7%，说明显式错误锚点定位对快速锁定异常区域具有决定性作用。去除实体回溯（w/o Entity Backtracking）时 Recall 与 F1 分别下降 12.6% 和 4.4%，表明该模块能有效补齐跨距离因果线索。去除语义压缩模型（w/o Semantic Compressor）后 Recall 不变而 Precision 下降 18.5%，说明轻量语义压缩主要贡献在于提高输出纯度与结构化摘要的稳定性。

## 4.4 根因分析实验结果

**Table 3:** 根因分析综合性能对比

| 分组 | 方法 | $s_i$ | $c_i$ | $r_i$ | $f_i$ | Score |
|---|---|---|---|---|---|---|
| DeepSeekV3 | RAG | 0.769 | 0.657 | 0.655 | 0.520 | 0.650 |
| | LogSage | 0.870 | 0.754 | 0.692 | 0.524 | 0.710 |
| | MCTS | 0.881 | 0.777 | 0.714 | 0.567 | 0.735 |
| | HRL-RCA | **0.892** | **0.798** | **0.766** | **0.706** | **0.790** |
| GPT-OSS | RAG | 0.743 | 0.625 | 0.693 | 0.613 | 0.669 |
| | LogSage | 0.791 | 0.699 | 0.721 | 0.606 | 0.704 |
| | MCTS | 0.841 | 0.747 | 0.754 | 0.590 | 0.733 |
| | HRL-RCA | **0.821** | **0.745** | **0.760** | **0.742** | **0.767** |
| 旗舰模型 | Seed 1.8 | 0.814 | 0.665 | 0.755 | 0.635 | 0.717 |
| | Kimi K2 | 0.833 | 0.713 | 0.744 | 0.592 | 0.721 |
| | DSV3.2 | 0.728 | 0.605 | 0.691 | 0.545 | 0.642 |

如表 3 所示，HRL-RCA 在两种基座下均取得最高综合得分。在 DeepSeekV3 基座下综合得分为 0.790，较最强基线 MCTS（0.735）提升 5.5%，较朴素 RAG（0.650）提升 21.5%。

如图 7 所示，HRL-RCA 的优势主要体现在修复推荐与根因解释环节：在 DeepSeekV3 基座下修复推荐得分达到 0.706，相比 MCTS 的 0.567 提升 0.139，说明通过历史修复规则的条件路由与证据绑定，更容易在满足适用前提的情况下召回可执行修复依据。关键错误提取得分在各方法中整体较高（0.769–0.892），说明统一的日志语义压缩已提供较强的错误锚点，而

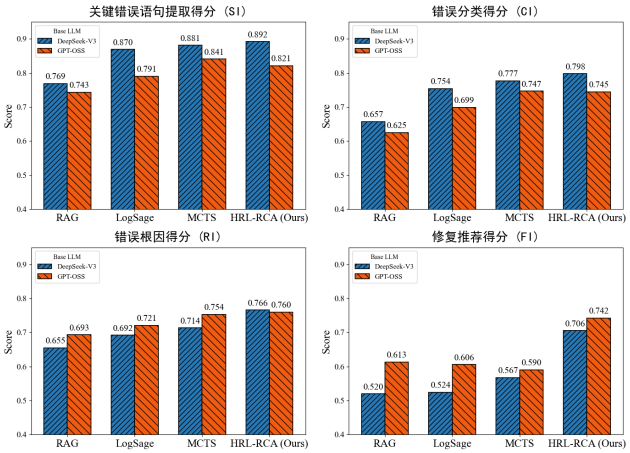


**Figure 7:** 各方法分项得分对比

HRL-RCA 仍保持最高（0.892），体现出结构化推理路径对推理稳定性的促进作用。

与独立旗舰推理模型基线相比，Seed1.8、Kimi K2 与 DeepSeekV3.2 的综合得分均低于 HRL-RCA，说明在构建失败这种强依赖适用前提与可验证证据的任务中，仅提升模型推理能力不足以稳定获得高质量修复建议，需要结构化的知识组织与证据约束机制。

## 4.5 效率分析

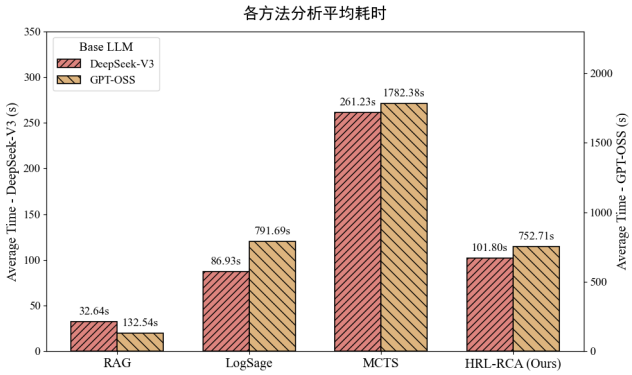


**Figure 8:** 各方法平均处理耗时对比

如图 8 所示，HRL-RCA 在 DeepSeekV3 基座下平均耗时 101.80s，相比 MCTS 的 261.23s 减少约 61.0%，与 LogSage（86.93s）基本持平。朴素 RAG 耗时最短（32.64s），但准确性显著不足。MCTS 因多轮搜索式推理导致耗时最长，而 HRL-RCA 通过规则路由将候选空间限制在局部范围，避免了全局搜索的开销，在保持最优准确性的同时兼顾了工程可用性。

## 5 总结与展望

本文面向 RISC -V 软件包构建失败场景，提出 "日志压缩 + 规则学习" 的两阶段根因分析方法。第一阶段 MGSC 通过显式错误锚点定位、基于实体的跨距离上下文回溯与轻量模型语义

压缩，在严格 token 预算下保留高保真证据链，显著降低长日志输入规模。第二阶段 HRL-RCA 将历史修复经验结构化为可判定规则并组织为树形索引，在线推理时通过条件路由召回满足前提的规则，在证据约束下生成可追溯的根因分析与修复建议。

实验结果表明，MGSC 在关键证据覆盖率上优于现有压缩方法，且输出 token 规模稳定可控；HRL-RCA 在综合诊断得分上达到 79.0%，显著超过朴素 RAG、多路检索增强与搜索式推理等基线，同时处理耗时较 MCTS 降低 61%，在效率上亦具有明显优势。未来工作将进一步完善规则知识库的增量更新与去重机制，探索将方法适配到更多架构与构建系统，并开发面向开发者的 IDE 插件等工具以促进工程落地。

## Acknowledgments

本工作得到中国科学院软件研究所等相关项目的支持。

## References

[1] 待补充. 2024. RCAgent: 待补充题目. 待补充 (2024). 占位条目，请补充真实文献信息。.
[2] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[3] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[4] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[5] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[6] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[7] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[8] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[9] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[10] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[11] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[12] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[13] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[14] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[15] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[16] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[17] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.
[18] 待补充. 2024. 待补充. 待补充 (2024). 占位条目，请补充真实文献信息。.